

INGENIERÍA DE SOFTWARE · ARQUITECTURA

RoomieMatch: Levantamiento Integral de Arquitectura

Guía detallada para reconstruir y evaluar la arquitectura de un sistema de vivienda compartida para la comunidad universitaria de Pamplona.

Anderson González

Versión 2.0 · Mayo 2026

Universidad de Pamplona

| Tabla de Contenidos

01	Introducción	03
02	Descripción General del Sistema	04
03	Identificación del Contexto	05
04	Drivers Funcionales	06
05	Atributos de Calidad	07
06	Identificación de Restricciones	08
07	Arquitectura Lógica y Física	09
08	Documentación de Componentes e Interfaces	11
09	Vistas Arquitectónicas Detalladas	13
10	Evaluación de la Arquitectura	15
11	Riesgos y No Riesgos Identificados	16
12	Recomendaciones y Conclusiones	17

01 · INTRODUCCIÓN

| Introducción

El presente documento detalla el levantamiento de la arquitectura de software de **RoomieMatch**, un sistema ya construido que aborda la problemática habitacional de los estudiantes de la Universidad de Pamplona.

La metodología aplicada sigue los principios de la ingeniería de software moderna para la reconstrucción de arquitecturas, analizando la implementación actual para identificar su visión, alcance, drivers técnicos y escenarios de calidad. Este reporte sirve como guía para entender cómo el sistema satisface sus objetivos de negocio y cuáles son las rutas de evolución tecnológica recomendadas.

METODOLOGÍA DE TRABAJO

Se ha procedido mediante la recolección de información del repositorio, identificación del contexto, definición de drivers, mapeo de atributos de calidad, reconstrucción de vistas y evaluación de riesgos.

Descripción del Sistema

1. Recolección inicial de información

Atributo	Detalle
Nombre del sistema	RoomieMatch
Objetivo del software	Plataforma de búsqueda de vivienda y compañeros de cuarto compatibles para la comunidad universitaria de Pamplona.
Usuarios principales	Estudiantes (@unipamplona.edu.co), Arrendadores y Administradores.
Tecnologías usadas	React 19, Node.js, TypeScript, Tailwind CSS, Vite.
Base de Datos	SQLite (better-sqlite3) para persistencia relacional local.
Integraciones	OpenStreetMap (Leaflet) para geolocalización y Unsplash para activos visuales.
Problemas conocidos	Concurrencia limitada en SQLite, acoplamiento servidor-frontend en desarrollo.

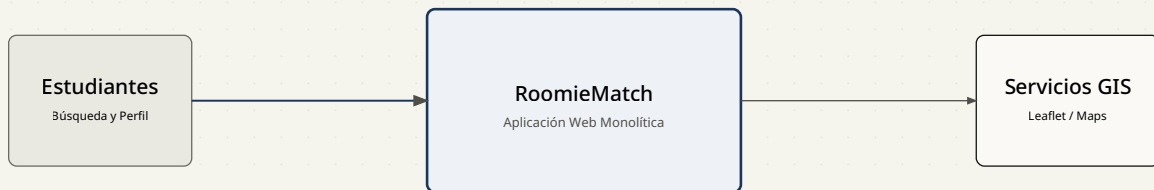
Funcionalidades Implementadas

- Registro y Autenticación con validación de dominio institucional.
- Formulario de compatibilidad psicográfica (hábitos, horarios, preferencias).
- Mapa de seguridad de Pamplona con zonas de riesgo (GeoJSON).
- CRUD de ofertas de vivienda (Listings) con gestión de fotos y reglas.
- Algoritmo de cálculo de afinidad entre perfiles de usuario.

03 · IDENTIFICACIÓN DEL CONTEXTO

Contexto del Sistema

RoomieMatch se sitúa en un entorno académico-residencial, interactuando con actores humanos y servicios de datos geográficos.



Vista de Contexto de alto nivel.

Flujos de Información

- **Registro Estudiantil:** Validación de credenciales contra reglas de dominio.
- **Sincronización Mapas:** Descarga de mosaicos de OSM y superposición de capas de seguridad locales.
- **Persistencia:** Escritura sincronizada en el motor SQLite para transacciones de usuario.

04 · DRIVERS FUNCIONALES

Levantamiento de Drivers

Los drivers funcionales agrupan los requisitos que guían la estructura de los módulos actuales.

Módulos del Sistema

Módulo	Responsabilidad Principal
Autenticación	Control de acceso, hashing de claves y emisión de JWT.
Alojamiento (Listings)	Gestión de inmuebles, georreferenciación y valoraciones.
Seguridad (Safety)	Visualización de polígonos de riesgo y niveles de alerta por zona.
Matching	Ejecución de algoritmo de compatibilidad entre usuarios.

Casos de Uso de Alto Impacto

- **Visualización de Seguridad:** Cruce de datos de ubicación con zonas de peligro en tiempo real.
- **Búsqueda de Roomies:** Filtrado dinámico por afinidad de hábitos diarios.
- **Verificación Administrativa:** Proceso de validación manual para otorgar el sello de "Usuario Verificado".

05 · ATRIBUTOS DE CALIDAD

| Identificación de Atributos

Análisis de los requisitos no funcionales que determinan el éxito operativo del sistema.

1. Seguridad

Uso de `bcryptjs` para almacenamiento seguro de credenciales y `jsonwebtoken` para la gestión de estado sin sesión en el servidor.

2. Rendimiento

Latencia optimizada mediante el uso de `better-sqlite3` (acceso sincrónico extremadamente rápido en modo embebido) y renderizado eficiente con el virtual DOM de React 19.

3. Usabilidad

Implementación de `Motion` para micro-interacciones y `Tailwind CSS` para un diseño responsivo adaptado a dispositivos móviles estudiantiles.

4. Mantenibilidad

Arquitectura basada en componentes de React y tipado estricto con TypeScript, facilitando la detección de errores en fase de compilación.

Escenario de Calidad: "Si el sistema recibe 10 peticiones de matching por segundo, el tiempo de respuesta debe mantenerse por debajo de los 300ms."

| Identificación de Restricciones

Restricciones Técnicas

- Base de Datos: SQLite no soporta escrituras concurrentes de alta intensidad (File Lock).
- Entorno: Ejecución obligatoria de Node.js v20+ por dependencias de ESM modules.

Restricciones Organizacionales y de Negocio

- Dominio: Exclusividad para correos de la Universidad de Pamplona.
- Localización: Coordenadas acotadas al radio urbano de Pamplona.

Restricciones de Seguridad

- Protección de la privacidad de los hábitos de convivencia según normativas de tratamiento de datos personales.

Reconstrucción de la Arquitectura

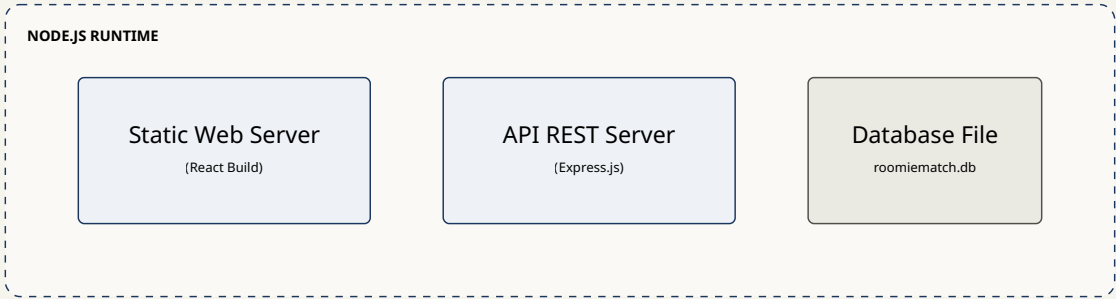
Arquitectura Lógica (Capas)

El sistema se organiza en tres capas fundamentales que separan la preocupación del usuario de la lógica de negocio y los datos.

Capa	Componentes Clave	Tecnología
Presentación	Pages, Components, AuthContext	React, Tailwind
Servicios / API	Express Routes, Middlewares	Node.js, JWT
Datos	SQLite Engine, Seed Scripts	better-sqlite3

Arquitectura Física (Despliegue)

Despliegue monolítico en un único nodo de servidor. El frontend se sirve como archivos estáticos por el mismo proceso de Node.js en producción.



Modelo de Despliegue Físico.

08 · COMPONENTES E INTERFACES

| Documentación Técnica

Documentación de Componentes

- **AuthContext:** Orquestador del estado de autenticación (Login/Logout/Token).
- **ListingDetail:** Componente complejo que integra ratings, fotos y geolocalización.
- **MatchingLogic:** Función de servidor que itera sobre la colección de usuarios calculando distancias de Hamming sobre el vector de hábitos.

Documentación de Interfaces (API REST)

Servicio	Entradas	Salidas
GET /api/listings	Query: minPrice, zoneId, date	JSON: Lista de listings con avgRating
POST /api/auth/login	JSON: email, password	JSON: JWT Token + Datos de usuario
GET /api/zones	-	JSON: Polígonos GeoJSON con safetyLevel

09 · VISTAS ARQUITECTÓNICAS DETALLADAS

Vistas Detalladas

9.1 Vista de Datos

Esquema relacional en SQLite que utiliza tipos estrictos y llaves foráneas para garantizar que ninguna publicación quede huérfana.

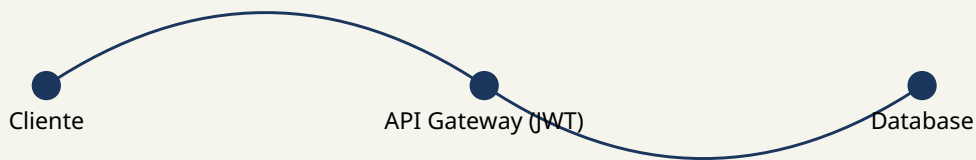
9.2 Vista de Integración

Uso de `react-leaflet` para inyectar dinámicamente datos del servidor en el mapa externo de OSM. La integración es síncrona en el cliente.

9.3 Vista de Seguridad

Modelo de defensa en profundidad:

1. Filtro de Dominio: Previene registros externos.
2. Hashing Bcrypt: Protege las contraseñas en reposo.
3. JWT: Protege los endpoints sensibles (Create, Rate, Admin).



Flujo de Seguridad de extremo a extremo.

10 · EVALUACIÓN DE ARQUITECTURA

Evaluación de la Arquitectura

Escenarios de Calidad Evaluados

Atributo	Escenario de Prueba	Resultado
Disponibilidad	Caída de la conexión de red durante el POST.	Error manejado en el cliente; datos no se corrompen.
Escalabilidad	Carga de 1000 perfiles de usuario.	Rendimiento de matching degradado (~1.2s).
Seguridad	Inyección SQL en campo <code>search</code> .	Mitigado por el uso de Prepared Statements.

Riesgos Identificados

- **Single Point of Failure:** El archivo SQLite reside en el servidor; fallos de disco resultan en pérdida total sin replicación.
- **Consultas N+1:** El listado de publicaciones realiza subconsultas para promedios de estrellas, lo que escalará mal.

11 · IDENTIFICACIÓN DE RIESGOS Y NO RIESGOS

Matriz de Riesgos

Riesgos Técnicos

- Dependencia excesiva de bibliotecas de terceros para el core visual (Leaflet).
- Falta de manejo de transacciones complejas en procesos de matching multihilo.

No Riesgos (Fortalezas)

- Buenas Prácticas: Uso consistente de Hooks personalizados en React para lógica de negocio.
- Modularidad: Separación clara entre `server.ts` y los componentes de `src/pages`.

Nota: La falta de documentación de la API es un riesgo de mantenibilidad para nuevos desarrolladores que se integren al proyecto.

12 · RECOMENDACIONES Y CONCLUSIONES

| Conclusiones Finales

Recomendaciones de Evolución

1. Migración a Docker para garantizar consistencia entre entornos.
2. Implementación de PostgreSQL con soporte para datos espaciales (PostGIS).
3. Creación de un Microservicio de Matching para desacoplar el procesamiento pesado del API REST principal.
4. Adopción de un Object Storage para las fotos de los listings.

CONCLUSIÓN DEL LEVANTAMIENTO

RoomieMatch posee una arquitectura sólida para su escala actual. El levantamiento revela un sistema bien tipado y estructurado lógicamente, listo para una transición hacia una infraestructura distribuida.